

RentEase - Rent Anything Safely!

Project Team

Abdullah Arif 20I-1762

Syed Muhammad Ali 20I-2303

Sher Muhammad 20I-0958

Session 2020-2024

Supervised by

Dr. Javaria

Co-Supervised by

Ms. Noor UL Ain



Department of Software Engineering

**National University of Computer and Emerging
Sciences Islamabad, Pakistan**

June, 2024

Contents

1	Introduction	1
1.1	Problem Statement.....	1
1.2	Scope.....	1
1.3	Modules.....	2
1.3.1	Module 1: User Management.....	2
1.3.2	Module 2: Product Listings.....	2
1.3.3	Module 3: Identity Verification.....	3
1.3.4	Module 4: Real-Time Communication.....	3
1.3.5	Module 5: Real-Time Tracking of Renters.....	3
1.3.6	Module 6: Blockchain-Based Proof of Ownership.....	3
1.3.7	Module 7: Rent-to-Own Model.....	3
1.4	User Classes and Characteristics.....	3
2	Project Requirements	5
2.1	Use-case/Event Response Table/Storyboarding.....	5
2.2	Functional Requirements.....	5
2.2.1	Module 1: User Management.....	5
2.2.2	Module 2: Product Listings.....	5
2.2.3	Module 3: Identity Verification.....	5
2.2.4	Module 4: Real-Time Communication.....	5
2.2.5	Module 5: Real-Time Tracking of Renters.....	5
2.2.6	Module 6: Blockchain-Based Proof of Ownership.....	5
2.2.7	Module 7: Rent-to-Own Model.....	5
2.3	Non-Functional Requirements.....	6
2.3.1	Reliability.....	6
2.3.2	Usability.....	6
2.3.3	Performance.....	6
2.3.4	Security.....	6
2.4	Domain Model.....	8
2.5	Use Cases.....	9
2.5.1	High-Level Use Cases for RentEase.....	9
2.5.2	Extended Use Cases for RentEase.....	12

2.6	Use Case Diagram.....	20
2.7	System Sequence Diagrams.....	21
3	System Overview	30
3.1	Architectural Design.....	31
3.2	Design Models.....	32
3.2.1	Activity Diagram.....	33
3.2.2	Class Diagram.....	34
3.2.3	Sequence Diagram.....	35
3.2.4	State Transition Diagram.....	35
3.2.5	JSON Schema.....	36
3.2.6	Data Flow Diagram.....	37
3.3	Data Design.....	37
3.3.1	Database Structure.....	37
3.3.2	Blockchain Data.....	38
3.3.3	Data Flow.....	38
3.4	Databases and Storage Items.....	39
	References	40

List of Figures

2.1	Domain Model	8
2.2	Use Case Diagram	20
2.3	Register-Login	21
2.4	Initiate rental process	22
2.5	Identity verification	23
2.6	Rental request approval	24
2.7	Real time communication	25
2.8	Track products	26
2.9	Manage payments	27
2.10	Report damage	28
2.11	Blockchain Base Transaction	29
3.1	Architecture Design	31
3.2	Box and Line Diagram	32
3.3	Activity Diagram	33
3.4	Class Diagram	34
3.5	Sequence Diagram	35
3.6	State Transition Diagram	35
3.7	JSON Schema Diagram	36
3.8	Data Flow Diagram	37

List of Tables

2.1	High-Level Use Cases for RentEase	9
2.2	Extended Use Cases for RentEase	12

Chapter 1

Introduction

This document specifies the requirements for the RentEase mobile application. RentEase is a secure and efficient platform that allows users to rent, co-own, and manage products using blockchain and machine learning technologies. The system is intended for developers, project managers, marketing staff, users, testers, and documentation writers, providing them with an in-depth view of the platform's functionality and modules.

1.1 Problem Statement

The current market lacks a secure platform for renting products. Most available systems fail to provide adequate security, identity verification, and transparent transaction histories. These issues result in mistrust among users, leading to losses and inefficiencies. RentEase addresses this gap by providing a transparent, trustable, and blockchain-enabled solution that ensures secure product rentals and ownership validation through advanced technologies such as machine learning and blockchain.

1.2 Scope

RentEase is designed to provide a comprehensive solution for renting and co-owning products. Its key features include:

- **User Management:** Users can create, modify, or delete their profiles and manage their ratings and reviews.
- **Product Listings:** Users can list products for rent, modify, or delete them, with predefined rental prices and durations.

- **Identity Verification:** RentEase incorporates multi-step verification using CNIC forensics, machine learning, and blockchain storage.
- **Real-Time Communication and Tracking:** The platform allows renters and owners to communicate and track product exchanges in real-time.
- **Shared Ownership:** Multiple users can co-own products, with rental fees divided among them.
- **Blockchain-Based Product Validation:** The blockchain is used to validate product ownership with unique serial numbers stored as digital tokens.
- **Assistant Chatbot:** The platform includes a chatbot to assist users with frequently asked questions.
- **Price Negotiation:** Renters and owners can negotiate rental terms and prices securely.

1.3 Modules

The RentEase application is composed of the following modules, which cover different system functionalities.

1.3.1 Module 1: User Management

The User Management module allows users to manage their profiles and their associated data.

1. Users can create, view, edit, and delete their profiles.
2. The system tracks ratings and reviews for each user.

1.3.2 Module 2: Product Listings

The Product Listings module allows users to manage their products.

1. Users can add, edit, or delete products from their listings.
2. Products include rental price, duration, and condition.

1.3.3 Module 3: Identity Verification

The Identity Verification module ensures secure identification of users.

1. Metadata and forensics check CNIC images for manipulations.
2. CNIC number extraction is validated with user input.
3. User CNIC and identity are stored in blockchain wallets.

1.3.4 Module 4: Real-Time Communication

The Real-Time Communication module facilitates communication between users.

1. Renters and owners can communicate through real-time chat.

1.3.5 Module 5: Real-Time Tracking of Renters

The Real-Time Tracking module ensures secure handoff of rented products.

1. Renter locations are tracked during product pickup and return.

1.3.6 Module 6: Blockchain-Based Proof of Ownership

This module verifies product ownership and tracks rental history.

1. Product ownership is verified using unique serial numbers stored in a blockchain ledger.

1.3.7 Module 7: Rent-to-Own Model

The Rent-to-Own module enables users to purchase products over time.

1. Ownership is transferred to the user after a specified rent amount is paid over time.

1.4 User Classes and Characteristics

RentEase has several key user classes:

User Class	Description
Renter	A Renter uses RentEase to rent products for short-term use. Renters can view available products, negotiate prices, and track products during pickup and return. They also communicate with owners through the platform.
Owner	An Owner lists products for rent. They manage product availability, negotiate prices, and track product handoff through real-time tracking. Owners can co-own products with other users.
Administrator	Administrators ensure the smooth running of the platform. They manage the blockchain system for product validation, verify user identities, resolve disputes, and handle the platform's overall operation.

Chapter 2

Project Requirements

This chapter details the functional and non-functional requirements for RentEase. The application is designed for secure, blockchain-based renting and co-ownership of products.

2.1 Use-case/Event Response Table/Storyboarding

For RentEase, the use case approach is ideal for gathering requirements. The platform involves interactions between renters, owners, and the blockchain for product ownership verification and transaction tracking.

2.2 Functional Requirements

The functional requirements are organized by module to provide a clear understanding of the capabilities and features of the RentEase application.

Module	Requirement Description
2.2.1 User Management	1. Profile Creation: Users must be able to create profiles using a valid email address and a secure password. 2. Profile Editing: Users must be able to update personal information such as name, contact number, and address. 3. Profile Deletion: Users should be able to delete their profiles permanently, with an option to confirm this action. 4. Ratings and Reviews: The system must allow users to leave ratings and reviews for transactions, visible to all users. 5. Password Reset: Users must be able to reset their passwords through a secure email verification process.
2.2.2 Product Listings	1. Product Creation: Users must be able to add new product listings with details like title, description, rental price, rental duration, and condition. 2. Product Editing: Users should be able to modify existing product details. 3. Product Deletion: Users must have the ability to remove their product listings. 4. Image Upload: Listings must allow users to upload multiple images of the product to enhance visibility. 5. Search Functionality: Users must be able to search for products based on categories, price range, and location.
2.2.3 Identity Verification	1. CNIC Verification: The system must verify user identities through a process involving scanning and analysis of CNIC documents. 2. Face Recognition: The application must utilize facial recognition technology to match user faces with their CNIC photos. 3. Blockchain Storage: Verified identities must be securely stored in blockchain wallets to

Module	Requirement Description
	prevent tampering. 4. Verification History: Users must have access to a history of their identity verification status and any updates. 5. Timely Processing: The identity verification process should be completed within a specified time frame (e.g., 5 minutes).
2.2.4 Real-Time Communication	1. Chat Functionality: Renters and owners must be able to initiate and maintain real-time chat conversations. 2. Message Notifications: Users should receive notifications for new messages to ensure timely responses. 3. File Sharing: The system must allow users to share images and documents during chats for better communication. 4. Chat History: Users must be able to access their chat history for reference. 5. Blocked Users: The system should enable users to block or report other users for inappropriate behavior.
2.2.5 Real-Time Tracking of Renters	1. Location Tracking: The system must track the geographical location of renters during product pickup and return. 2. Real-Time Updates: Users should receive updates about the location and estimated time of arrival (ETA) for pickups in real time. 3. Tracking History: The application must maintain a history of tracking data for future reference by both renters and owners. 4. Privacy Controls: Users must be able to enable or disable location tracking at any time. 5. Notifications: Users should be notified when a rental is confirmed, including pickup and return details.
2.2.6 Blockchain-Based Proof of Ownership	1. Ownership Verification: The system must verify product ownership through blockchain technology to ensure authenticity. 2. Transaction History Storage: All transactions related to product rentals must be securely stored on the blockchain for transparency. 3. Ownership Transfer: Users should be able to transfer ownership of a product through a secure blockchain transaction. 4. Ownership History Access: Users must be able to view the complete ownership history of a product before renting. 5. Dispute Resolution: The system should provide a mechanism for resolving disputes regarding product ownership.
2.2.7 Rent-to-Own Model	1. Payment Plan Creation: Users must be able to set up payment plans for rent-to-own transactions, including payment amounts and schedules. 2. Ownership Transfer: Users must be able to transfer ownership of products after fulfilling rent payments, with clear documentation of completed payments. 3. Remaining Balance View: Users should be able to view their remaining balance and payment history within their profiles. 4. Notifications for Payments: The system must send reminders for upcoming payments and confirmations upon successful payments. 5. Flexible Payment Options: Users must have multiple options for making payments, including credit cards and digital wallets.

2.3 Non-Functional Requirements

The non-functional requirements outline the quality attributes and constraints of the RentEase application.

Requirement	Description
2.3.1 Reliability	- RentEase must ensure high availability, with a target uptime of 99.9%. - The platform must handle large volumes of concurrent transactions without downtime. - The system should provide backup and recovery solutions to prevent data loss.
2.3.2 Usability	- The system should provide an intuitive user interface that accommodates users from diverse backgrounds, including those with limited technical skills. - Clear instructions and tooltips must guide users through key features such as user management, product listings, and communication. - The application must support accessibility features to assist users with disabilities.
2.3.3 Performance	- Pages should load within 2 seconds for users with a 20 Mbps internet connection to ensure a smooth user experience. - Real-time communication and tracking data should update within 1 second. - The system must support at least 10,000 concurrent users without performance degradation.
2.3.4 Security	- RentEase must ensure the protection of user data through encryption during data transmission and storage. - The application must comply with relevant data protection regulations, such as GDPR. - The system should implement multi-factor authentication (MFA) for user accounts to enhance security. - Regular security audits must be conducted to identify and mitigate vulnerabilities.
2.3.5 Scalability	- The system must be designed to handle increased loads, with the ability to scale resources up or down as needed. - The architecture should support the addition of new features and modules without significant rework.
2.3.6 Maintainability	- The codebase must be well-documented to facilitate future maintenance and updates. - The application should be built using modular components to simplify testing and debugging. - Regular software updates must be planned to address bugs and introduce new features.

2.3 Domain Model

The RentEase domain model defines the key relationships between users, products, trans- actions, and the blockchain. The system uses blockchain to manage the ownership and transaction history of products while maintaining secure and reliable data exchange.

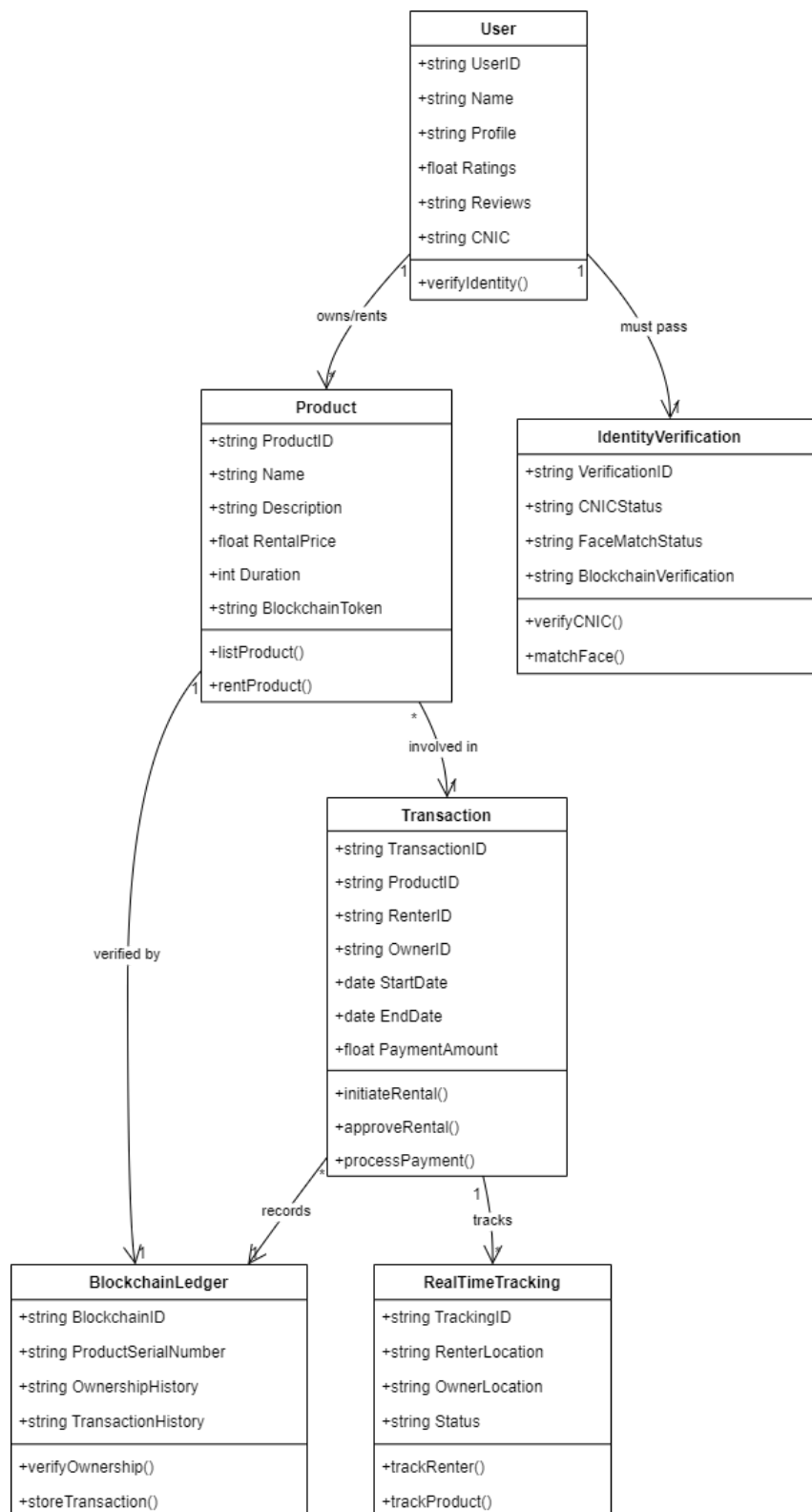


Figure 2.1: Domain Model

2.5 Use Cases

Use Case: View and Manage Product Listings

Use Case ID	UC-1
Use Case	View and Manage Product Listings
Actor	Owner, Renter
Type	Primary
Description	Owners can create, edit, and delete product listings, while renters can view available products.
Preconditions	The owner must be authenticated to list a product.
Postconditions	The product listing is updated and available for viewing by potential renters.

Use Case: Initiate Rental Process

Use Case ID	UC-2
Use Case	Initiate Rental Process
Actor	Renter, Owner
Type	Primary
Description	Renters initiate a rental request for a product listed by an owner.
Preconditions	The renter must be logged in and have selected a product.
Postconditions	The rental request is sent to the owner.

Use Case: Identity Verification

Use Case ID	UC-3
Use Case	Identity Verification
Actor	Renter, System
Type	Secondary
Description	The system verifies the user's identity using CNIC forensics, facial matching, and blockchain-based verification.
Preconditions	The user must provide a CNIC image and a live photo.
Postconditions	The identity is verified, or the rental process is halted if the identity check fails.

Use Case: Approve Rental Request

Use Case ID	UC-4
Use Case	Approve Rental Request
Actor	Owner, Renter
Type	Primary
Description	The product owner reviews and approves or denies a rental request.
Preconditions	The renter has submitted a valid rental request.
Postconditions	The rental is approved, and the terms are confirmed.

Use Case: Real-Time Communication

Use Case ID	UC-5
Use Case	Real-Time Communication
Actor	Owner, Renter
Type	Primary
Description	Owners and renters can communicate in real time to discuss rental details.
Preconditions	A rental request has been initiated.
Postconditions	Both parties can exchange messages regarding the rental.

Use Case: Track Product and Renter

Use Case ID	UC-6
Use Case	Track Product and Renter
Actor	Renter, Owner, System
Type	Secondary
Description	Real-time tracking of the renter's location during the product pickup and return process.
Preconditions	A rental transaction is in progress.
Postconditions	The renter's live location is shared with the owner.

Use Case: Manage Payment and Rewards

Use Case ID	UC-7
Use Case	Manage Payment and Rewards
Actor	Renter, System
Type	Primary
Description	Process payments for rentals using credit/debit cards or cash, and manage rewards for users.
Preconditions	A rental agreement has been finalized.
Postconditions	Payment is processed, and the user receives any applicable rewards.

Use Case: Blockchain-Based Ownership Validation

Use Case ID	UC-8
Use Case	Blockchain-Based Ownership Validation
Actor	System, Blockchain
Type	Secondary
Description	The system verifies product ownership through blockchain validation.
Preconditions	The product must be listed with a valid serial number and blockchain token.
Postconditions	Ownership is validated, and the product is associated with a blockchain entry.

Use Case: Report and Resolve Product Damage

Use Case ID	UC-9
Use Case	Report and Resolve Product Damage
Actor	Renter, Owner, System
Type	Primary
Description	Users report product damages, and the system manages resolution through a dedicated center.
Preconditions	The product has been rented and returned with damage.
Postconditions	The issue is resolved, and any compensation is handled.

2.5.1 Extended Use Cases for RentEase

Extended Use Case: View and Manage Product Listings

Designation	UC-1
Name	View and Manage Product Listings
Description	Owners can create, edit, and delete product listings, while renters can view available products.
Trigger Event	User wants to manage product listings (for owners) or view available listings (for renters).
Actors	Owner, Renter
Precondition	The owner must be authenticated to list a product.
Postcondition	The product listing is updated and available for viewing by potential renters.
Main Scenario	1. Owner logs into the system. 2. Owner navigates to product management. 3. Owner creates/edits/deletes a product. 4. Renter searches and views the updated listings.
Alternate Scenario	If the product listing is already rented, the system prevents the owner from editing or deleting it.
Exception Scenario	If the owner is not authenticated, they are prompted to log in.

Extended Use Case: Initiate Rental Process

Designation	UC-2
Name	Initiate Rental Process
Description	Renters initiate a rental request for a product listed by an owner.
Trigger Event	The renter selects a product to rent.
Actors	Renter, Owner
Precondition	The renter must be logged in and have selected a product.
Postcondition	The rental request is sent to the owner.
Main Scenario	1. Renter logs into the system. 2. Renter browses the available product listings. 3. Renter selects a product and initiates the rental process. 4. The owner receives the rental request.
Alternate Scenario	If the product is unavailable (already rented), the system informs the renter and suggests other products.
Exception Scenario	If the renter is not logged in, they are prompted to log in before proceeding with the rental process.

Extended Use Case: Identity Verification

Designation	UC-3
Name	Identity Verification
Description	The system verifies the user's identity using CNIC forensics, facial matching, and blockchain-based verification.
Trigger Event	A user attempts to initiate a rental transaction or add a product listing.
Actors	Renter, System
Precondition	The user must provide a CNIC image and a live photo.
Postcondition	The identity is verified, or the rental process is halted if the identity check fails.
Main Scenario	1. The system prompts the user to provide a CNIC and a live photo. 2. The system analyzes the CNIC using forensics. 3. The system matches the live photo with the CNIC. 4. The system verifies the user's identity on the blockchain.
Alternate Scenario	If the CNIC verification fails, the system requests the user to re-upload documents or try an alternate verification method.
Exception Scenario	If the identity cannot be verified after multiple attempts, the system halts the rental process and flags the account for review.

Extended Use Case: Approve Rental Request

Designation	UC-4
Name	Approve Rental Request
Description	The product owner reviews and approves or denies a rental request.
Trigger Event	The renter sends a rental request to the owner.
Actors	Owner, Renter
Precondition	The renter has submitted a valid rental request.
Postcondition	The rental is approved, and the terms are confirmed.
Main Scenario	1. Owner receives a rental request. 2. Owner reviews the renter's profile and product availability. 3. Owner approves or denies the request. 4. The system confirms the rental terms.
Alternate Scenario	If the owner does not respond within a predefined time, the rental request expires or is automatically approved.
Exception Scenario	If the owner attempts to approve a request for an unavailable product, the system notifies the owner and cancels the request.

Extended Use Case: Real-Time Communication

Designation	UC-5
Name	Real-Time Communication
Description	Owners and renters can communicate in real time to discuss rental details.
Trigger Event	The rental request has been initiated, and both parties wish to communicate.
Actors	Owner, Renter
Precondition	A rental request has been initiated.
Postcondition	Both parties can exchange messages regarding the rental.
Main Scenario	1. Renter sends a rental request. 2. The owner and renter engage in real-time chat through the system. 3. Both parties discuss rental terms and logistics.
Alternate Scenario	If one party is unavailable, the system allows for message notifications and delayed replies.
Exception Scenario	If the chat feature is unavailable, the system provides an email contact option.

Extended Use Case: Track Product and Renter

Designation	UC-6
Name	Track Product and Renter
Description	Real-time tracking of the renter's location during the product pickup and return process.
Trigger Event	A rental transaction is in progress, and the owner or system wishes to track the renter's location.
Actors	Renter, Owner, System
Precondition	A rental transaction is in progress.
Postcondition	The renter's live location is shared with the owner.
Main Scenario	1. The renter picks up the product. 2. The system starts tracking the renter's location. 3. The owner views the live location. 4. The product is returned and tracking is stopped.
Alternate Scenario	If tracking fails, the system notifies both parties and advises contacting support.
Exception Scenario	If the renter enters a restricted area, the system sends alerts and logs the event.

Extended Use Case: Manage Payment and Rewards

Designation	UC-7
Name	Manage Payment and Rewards
Description	Process payments for rentals using credit/debit cards or cash, and manage rewards for users.
Trigger Event	The rental agreement is finalized, and payment needs to be processed.
Actors	Renter, System
Precondition	A rental agreement has been finalized.
Postcondition	Payment is processed, and the user receives any applicable rewards.
Main Scenario	1. Renter selects a payment method. 2. System processes the payment. 3. Payment is confirmed, and rewards (if applicable) are applied.
Alternate Scenario	If the primary payment method fails, the system requests an alternate method.
Exception Scenario	If payment cannot be processed, the system notifies the renter and cancels the rental transaction.

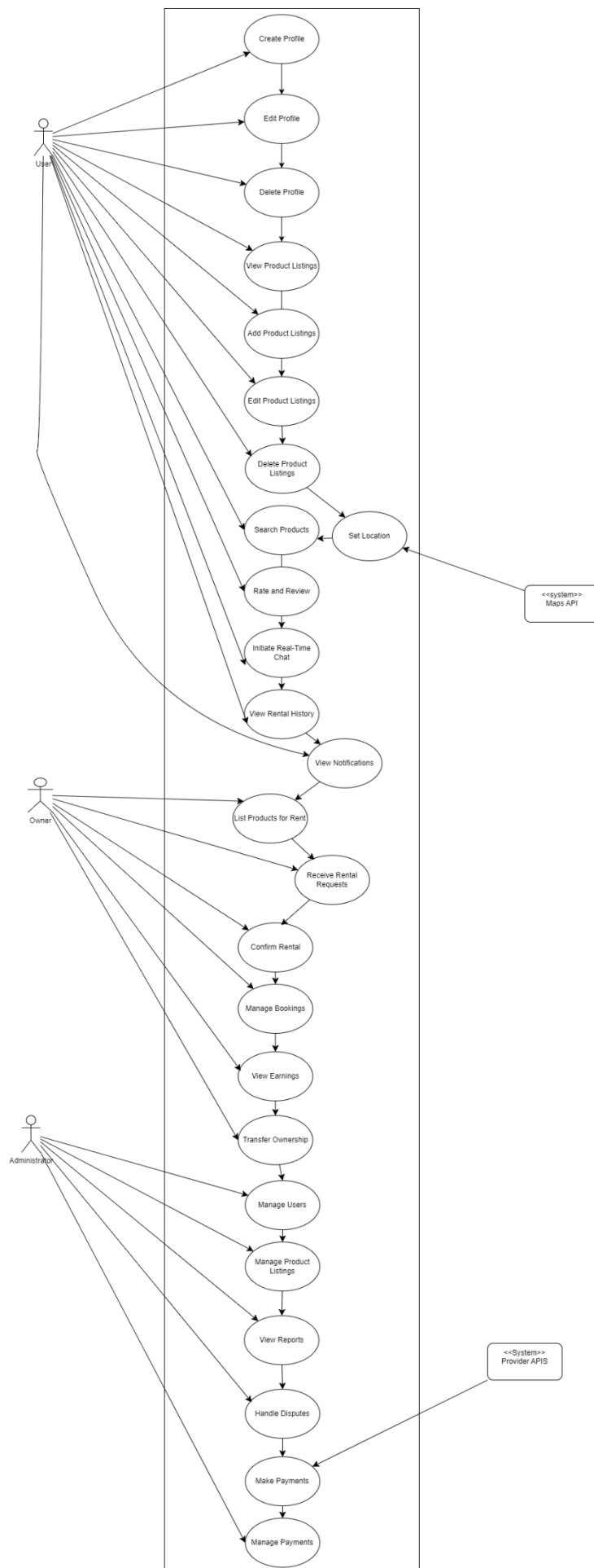
Extended Use Case: Blockchain-Based Ownership Validation

Designation	UC-8
Name	Blockchain-Based Ownership Validation
Description	The system verifies product ownership through blockchain validation.
Trigger Event	The owner lists a product for rent, and ownership needs to be validated.
Actors	System, Blockchain
Precondition	The product must be listed with a valid serial number and blockchain token.
Postcondition	Ownership is validated, and the product is associated with a blockchain entry.
Main Scenario	1. The owner lists the product. 2. The system retrieves the blockchain token. 3. The system validates the ownership via blockchain. 4. The system confirms the product's ownership and adds it to the platform.
Alternate Scenario	If blockchain validation fails, the system retries validation using a different blockchain node.
Exception Scenario	If ownership cannot be validated, the system flags the product for review and prevents it from being listed.

Extended Use Case: Report and Resolve Product Damage

Designation	UC-9
Name	Report and Resolve Product Damage
Description	Users report product damages, and the system manages resolution through a dedicated center.
Trigger Event	A product is returned damaged by the renter.
Actors	Renter, Owner, System
Precondition	The product has been rented and returned with damage.
Postcondition	The issue is resolved, and any compensation is handled.
Main Scenario	1. Renter reports the product damage. 2. The system notifies the owner. 3. The owner reviews the damage. 4. The system manages the dispute resolution and compensation (if necessary).
Alternate Scenario	If the renter disputes the damage, the system engages a third-party review process.
Exception Scenario	If the damage is disputed but cannot be resolved, the case is escalated to customer service.

2.6 Use Case Diagram



2.5 System Sequence Diagrams

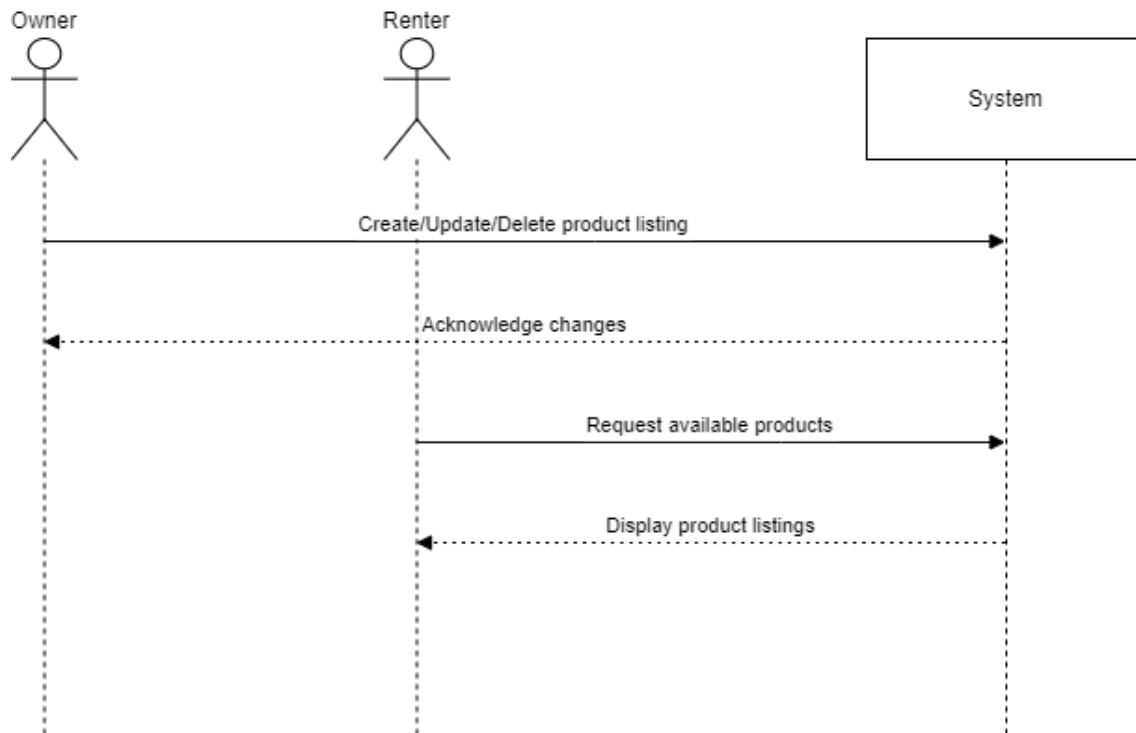


Figure 2.4: Manage product listings

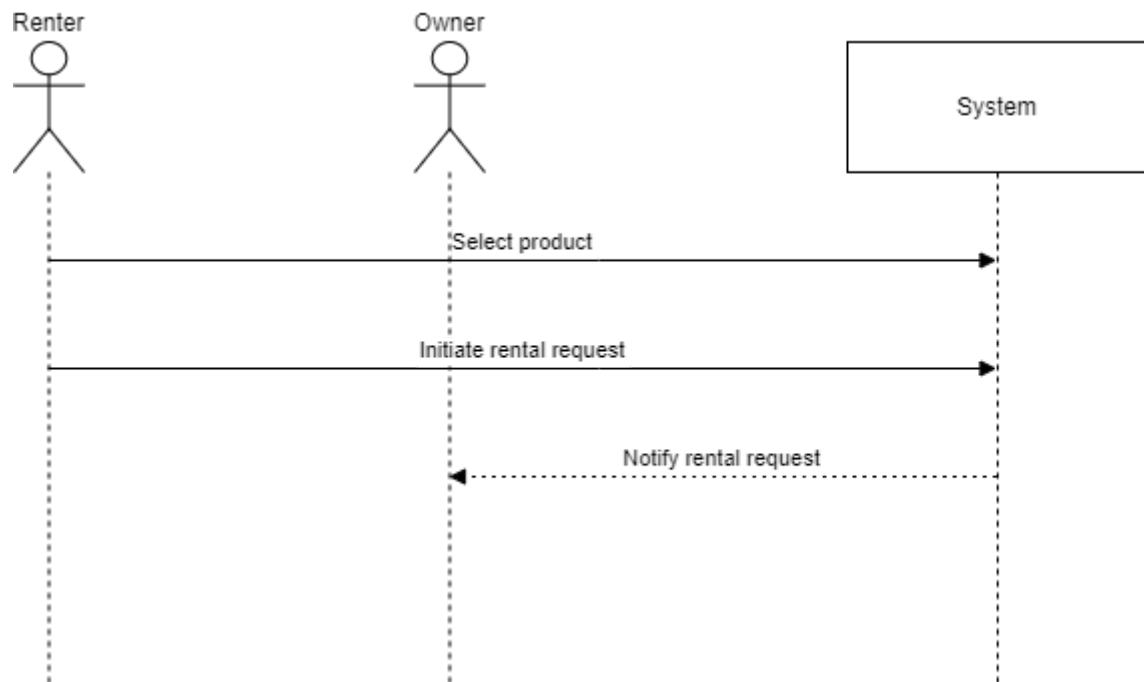


Figure 2.5: Initiate rental process

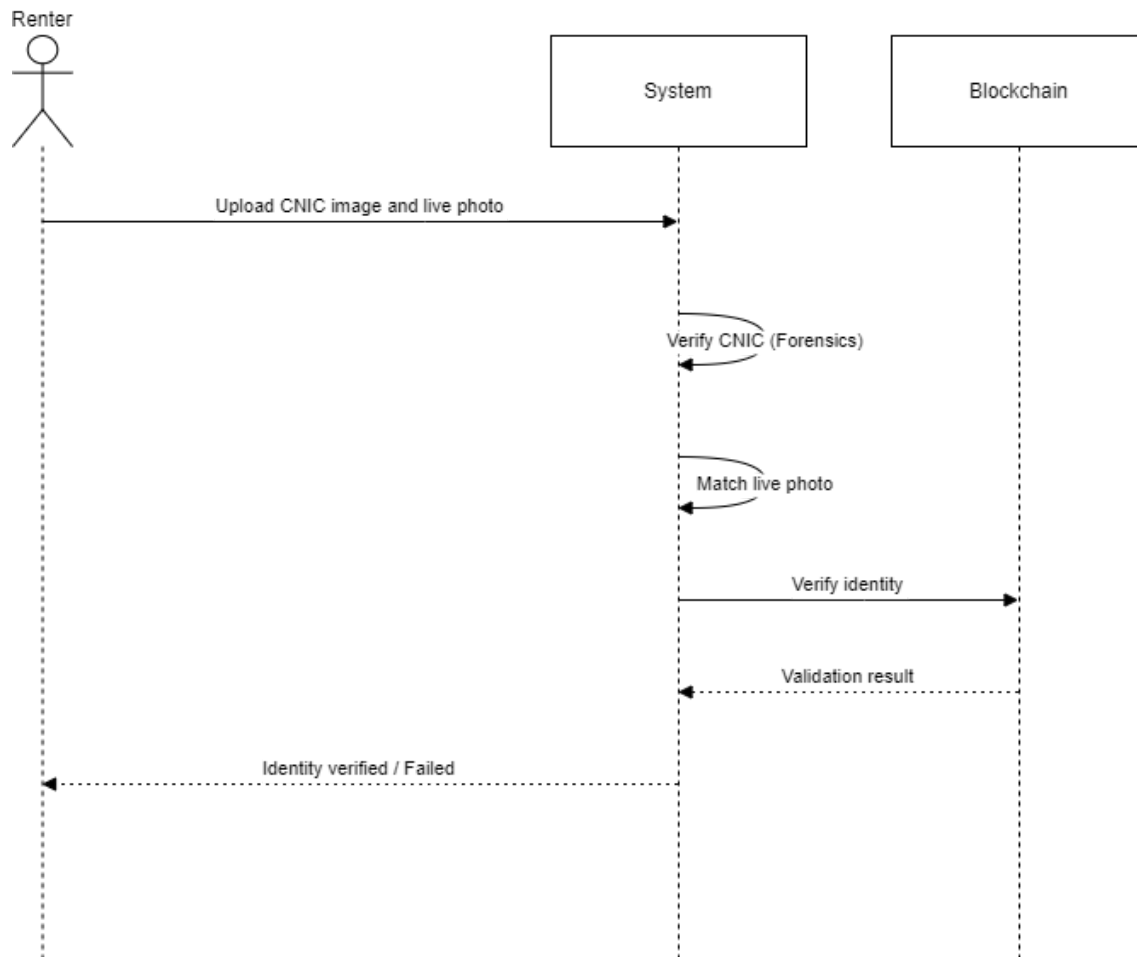


Figure 2.6: Identity verification

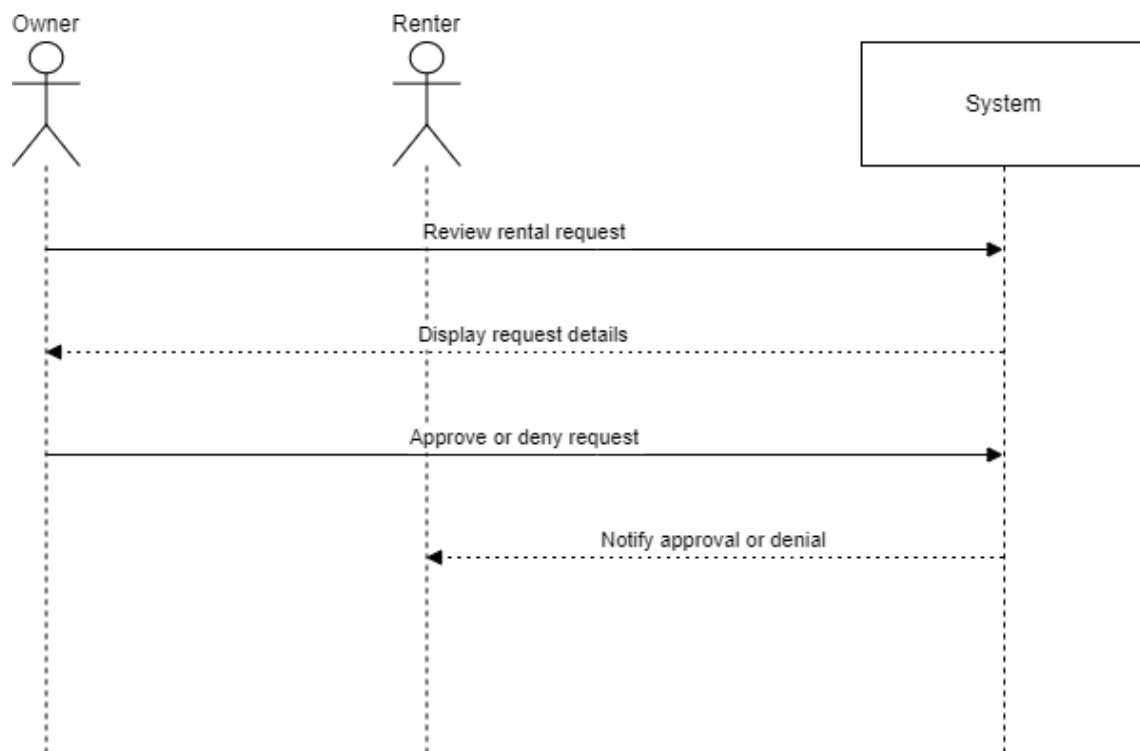


Figure 2.7: Rental request approval

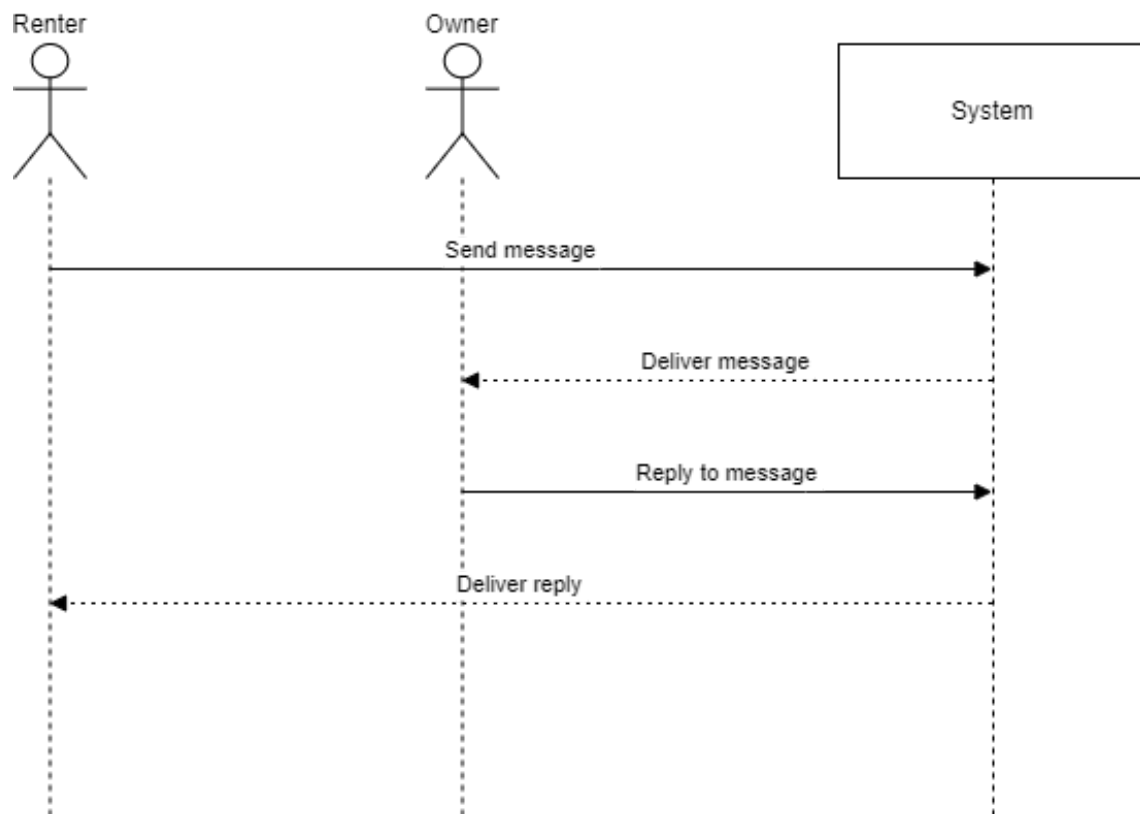


Figure 2.8: Real time communication

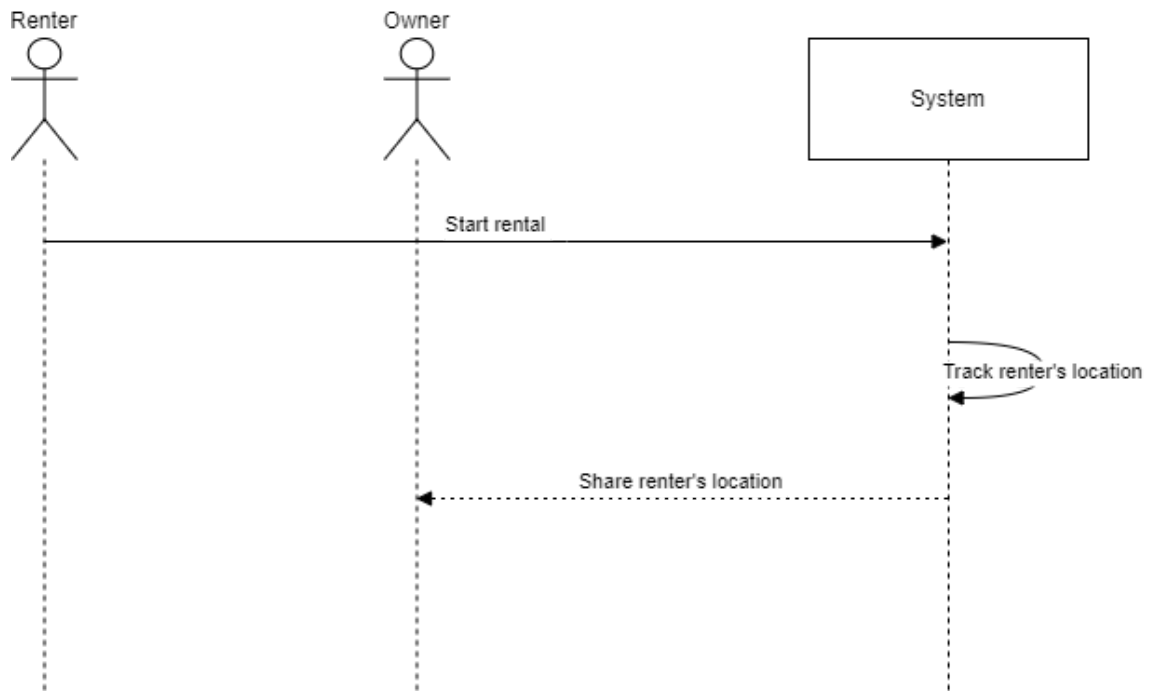


Figure 2.9: Track products

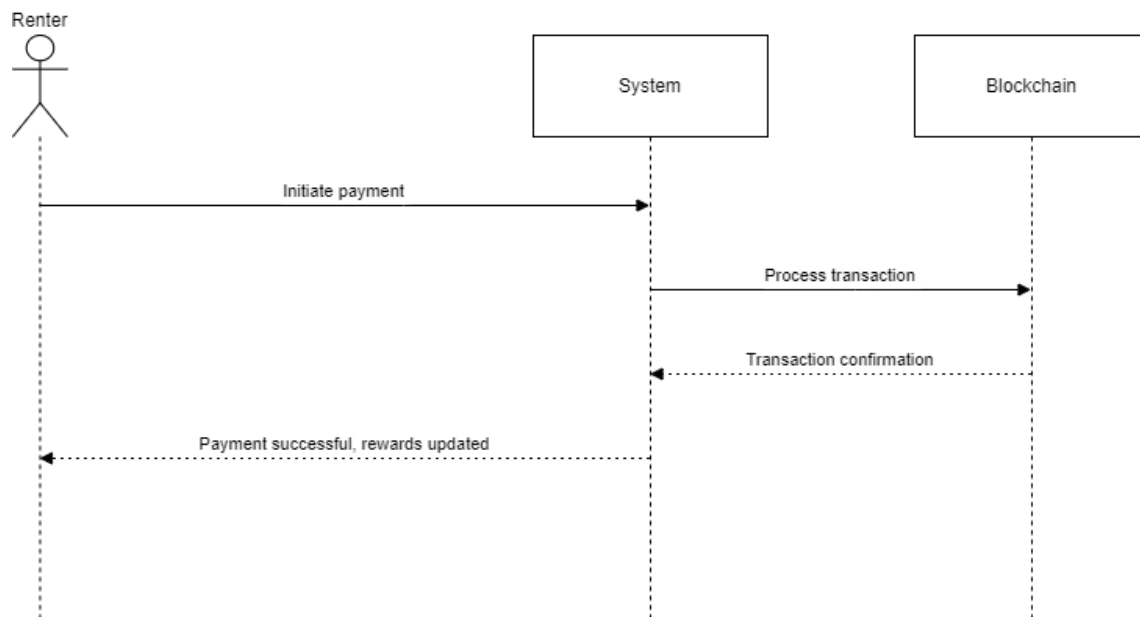


Figure 2.10: Manage payments

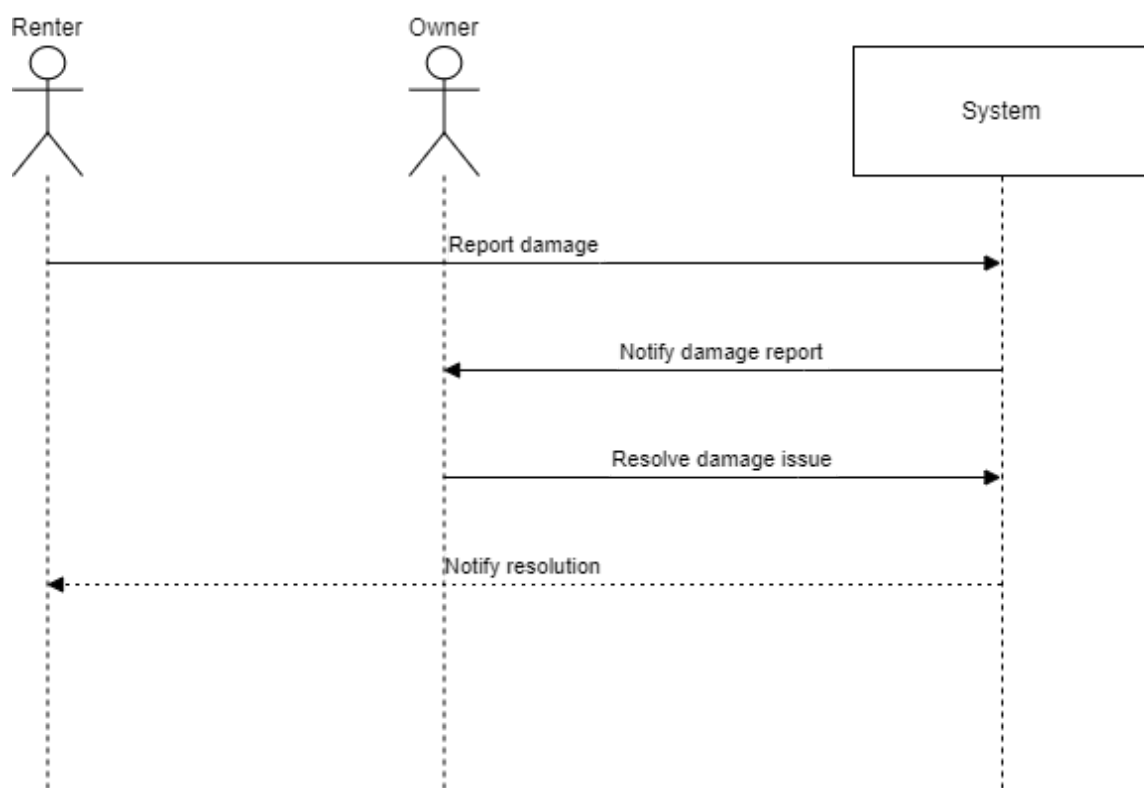


Figure 2.11: Report damage

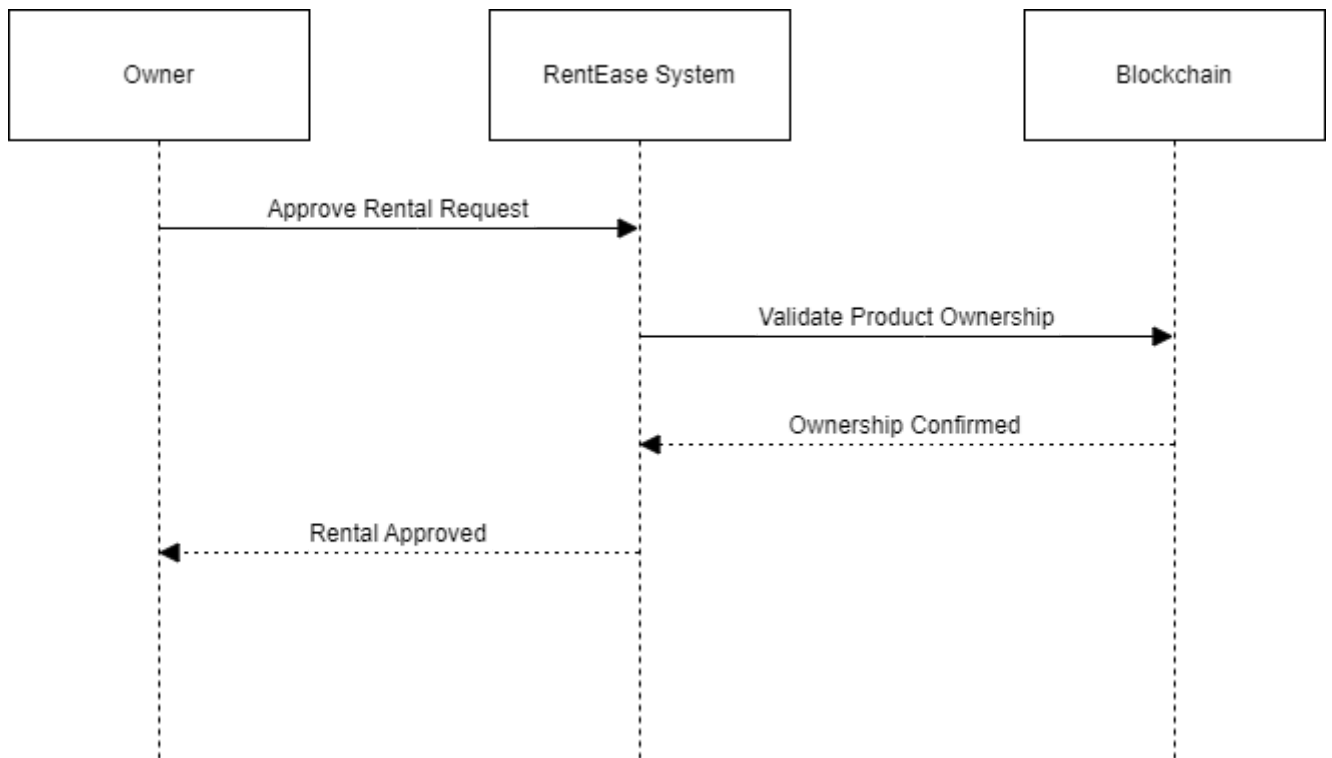


Figure 2.15: Blockchain-Based Ownership Validation

Chapter 3

System Overview

The RentEase platform is a mobile application designed to facilitate secure and transparent product rentals between users, utilizing blockchain and machine learning technologies. The primary goal of the system is to ensure a trust-based environment where users (renters and owners) can exchange products with validated identity checks, real-time communication, product tracking, and ownership validation through blockchain.

The system is structured into modular components that collectively deliver the full functionality of the platform. Each component plays a distinct role but interacts with other subsystems to provide a seamless experience for users. Key features of the system include user management, product listings, identity verification, real-time tracking, price negotiation, and product validation through blockchain.

3.1 Architectural Design

The architecture of the RentEase system follows a **Client-Server** model combined with a **Layered Architecture**. The application is decomposed into distinct layers, each serving a particular role in the system, such as frontend (user interaction), backend (business logic), database management, and blockchain integration.

The system's design is modular, with each module handling specific functionalities:

- **Frontend (Client Layer):** The mobile interface built using React Native.
- **Backend (Server Layer):** The server-side logic and APIs, implemented using Node.js.
- **Database Layer:** The layer where user data, product listings, and transaction details are stored. Uses MongoDB and Firebase for database management.

- **Blockchain Layer:** Handles decentralized product ownership and identity verification using Ethereum blockchain.

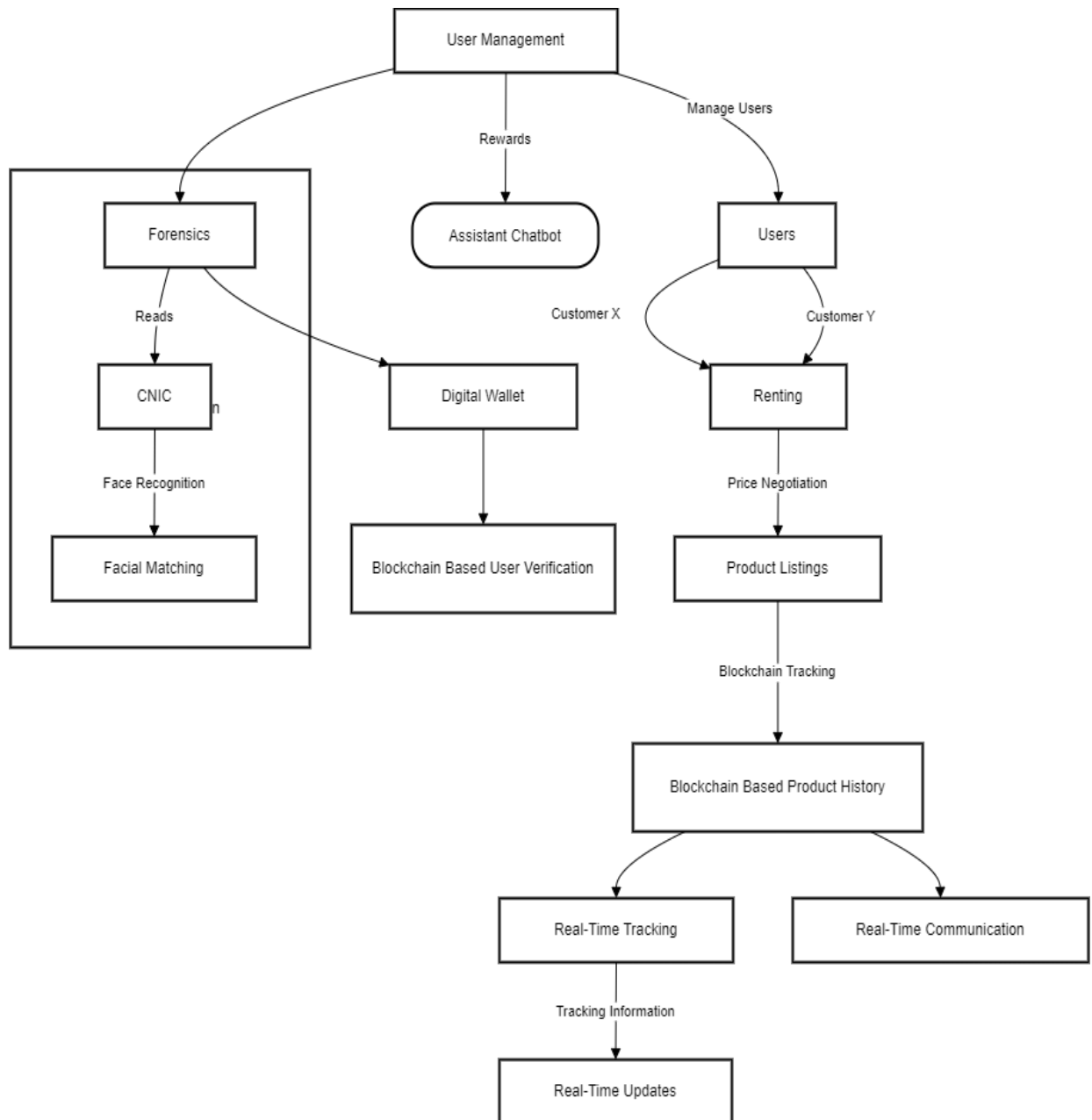


Figure 3.1: Architecture Design

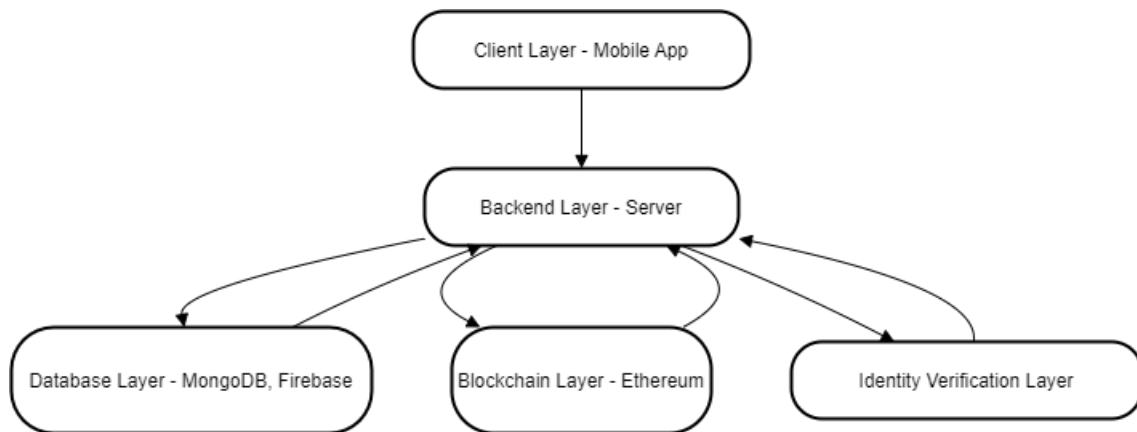


Figure 3.2: Box and Line Diagram

3.2 Design Models

Below are the various design models that describe the architecture and behavior of the RentEase system.

3.2.1 Activity Diagram

The activity diagram models the flow of actions in the system, such as the process of renting a product or verifying a user's identity.

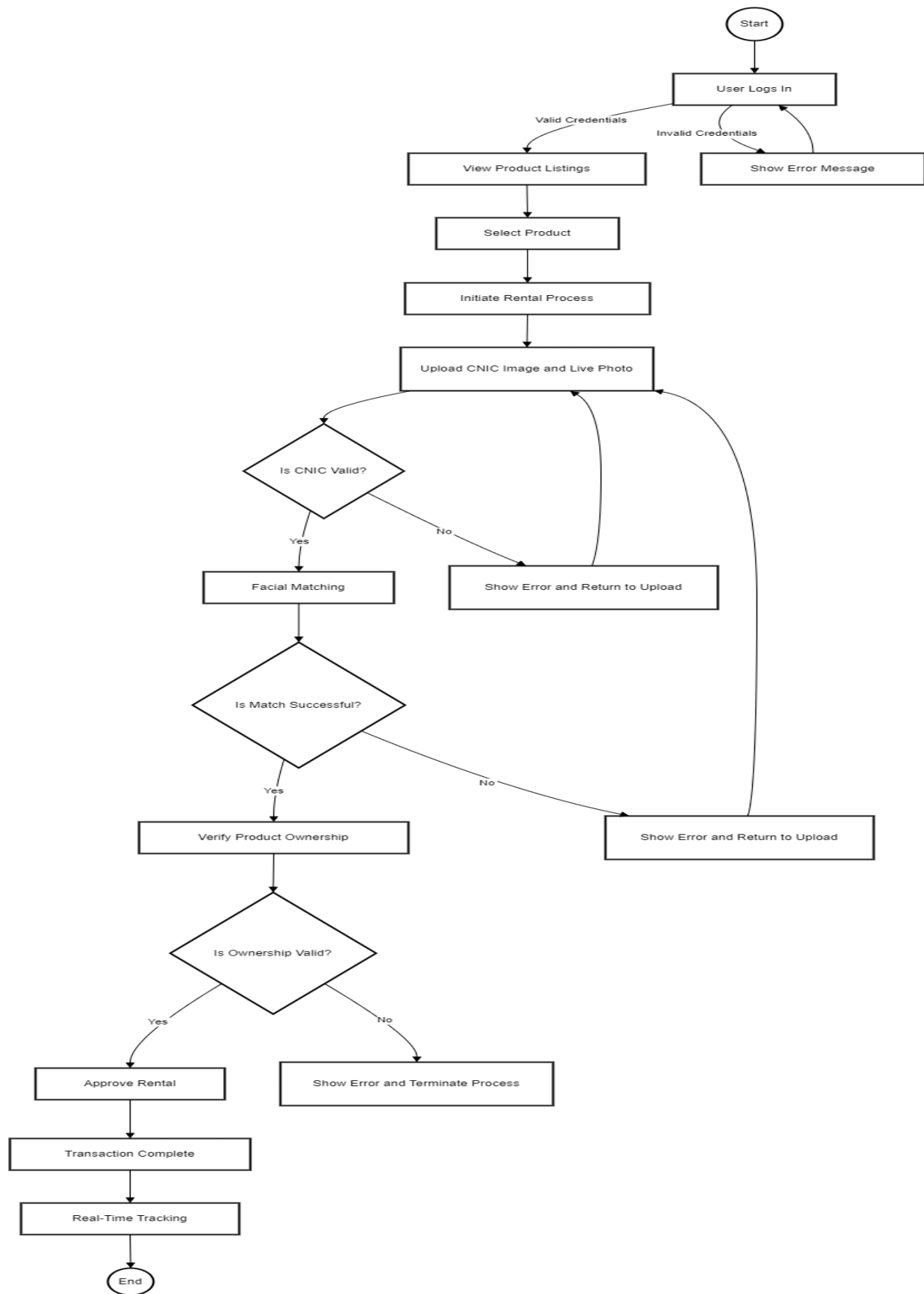


Figure 3.3: Activity Diagram

3.2.1 Class Diagram

The class diagram is a structural model that defines the main entities in the system, such as ‘User‘, ‘Product‘, ‘Transaction‘, and their relationships

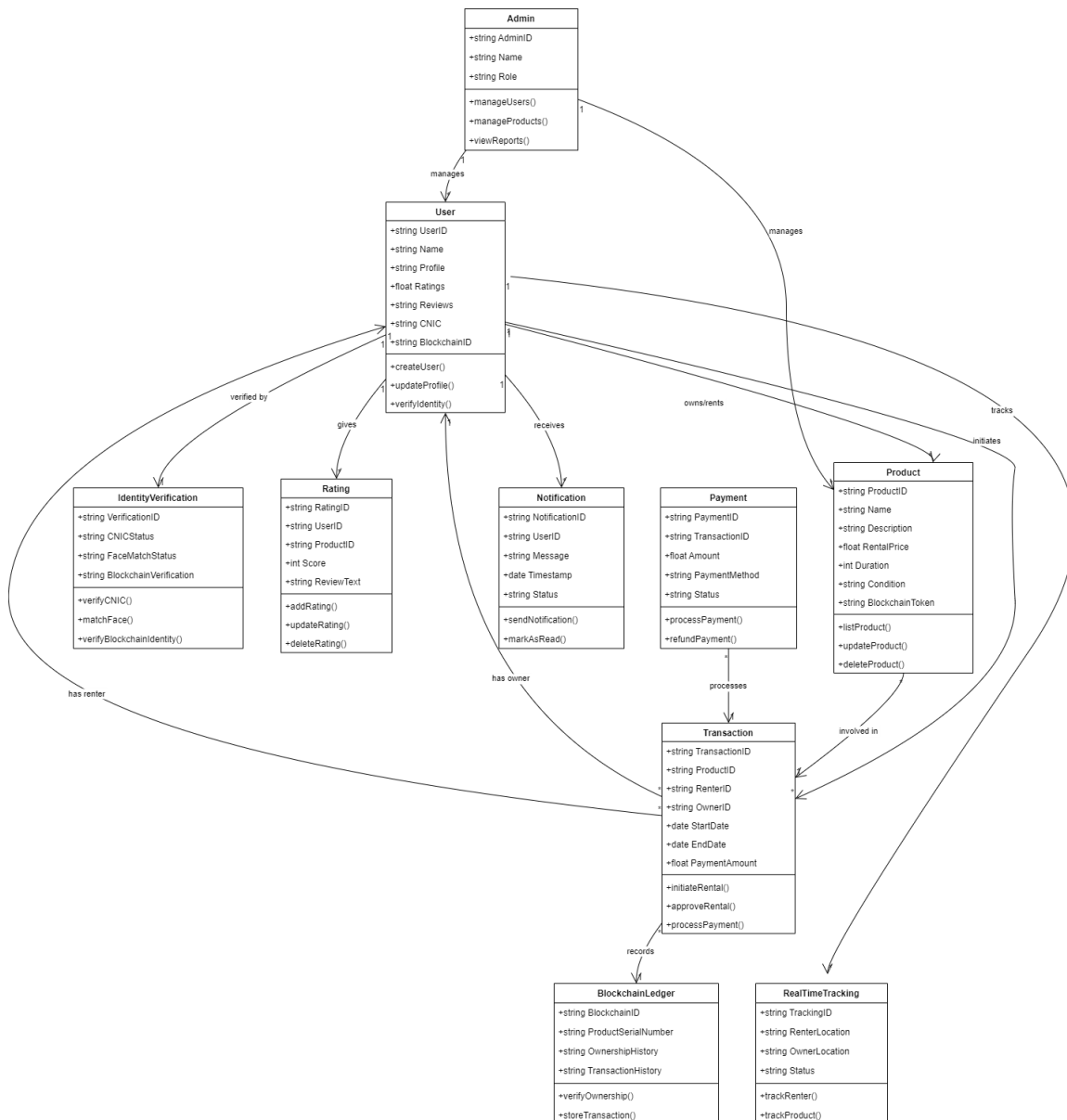


Figure 3.4: Class Diagram

3.2.1 Sequence Diagram

A sequence diagram models the interactions between objects in a particular sequence. Below is a diagram showing the sequence of interactions between a user (renter), product, and the blockchain during the product rental process.

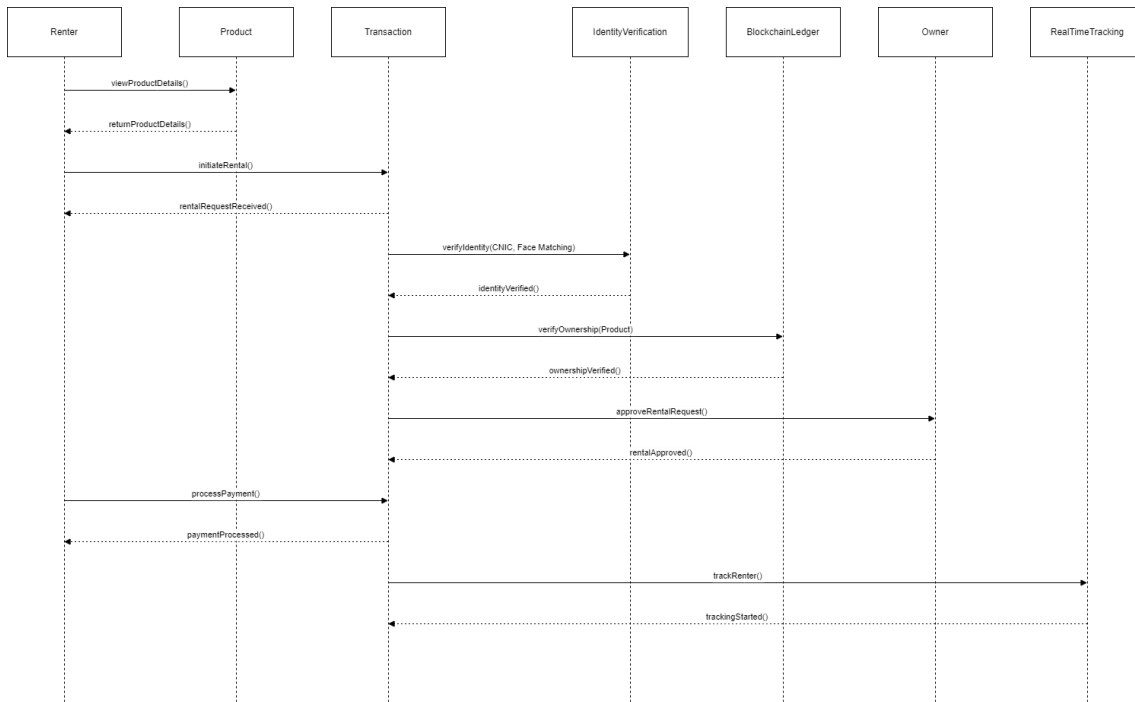


Figure 3.5: Sequence Diagram

3.2.2 State Transition Diagram

For processes such as user verification and transaction management, state transition diagrams depict how the system moves between states based on events or user actions.

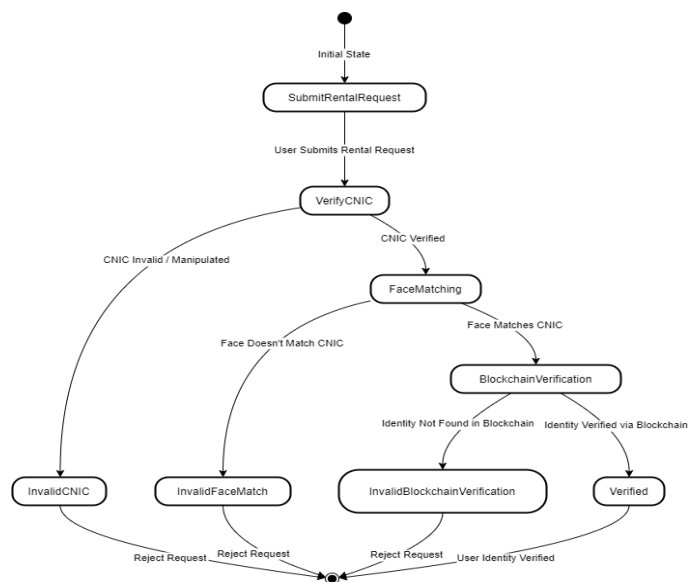
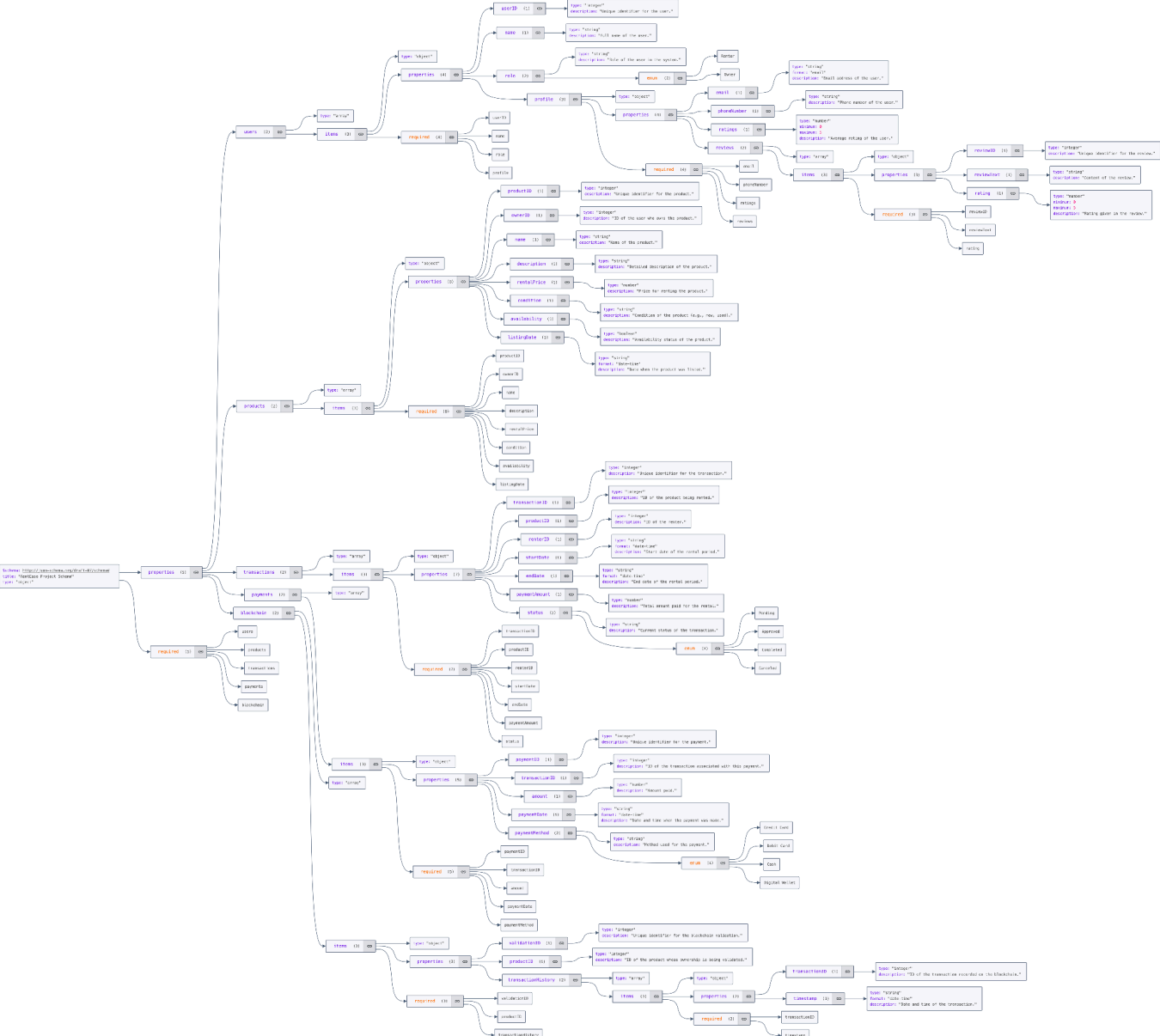
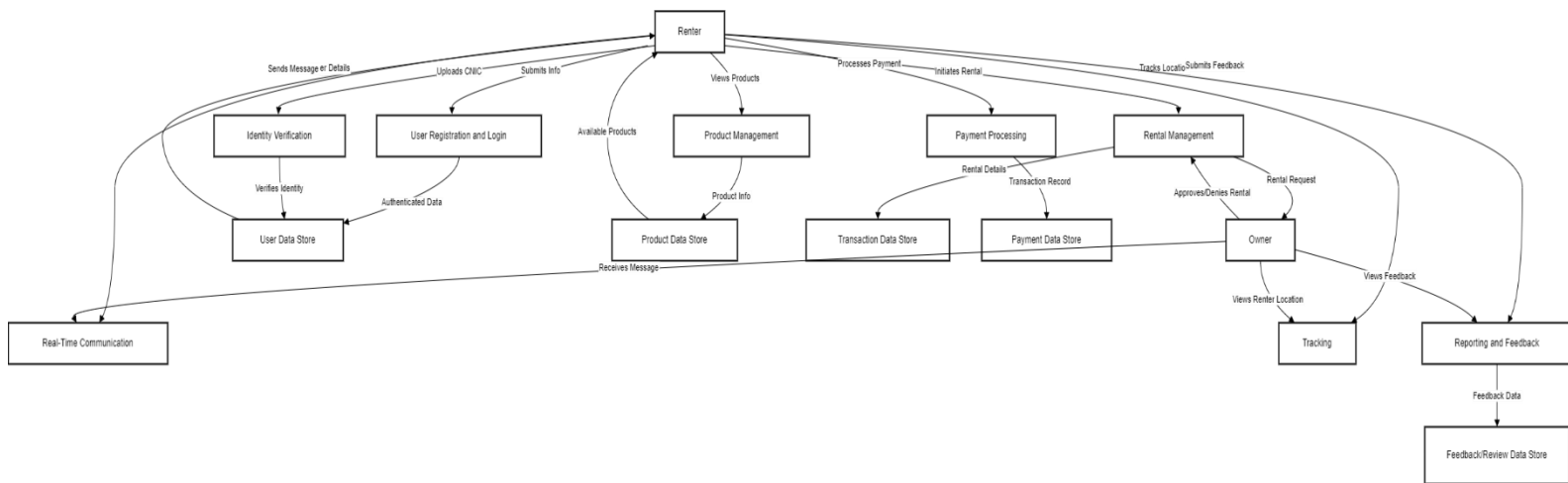


Figure 3.6: State Transition Diagram



3.2.4 Data Flow Diagram



3.3 Data Design

The RentEase platform processes and stores several types of data, including user profiles, product listings, transactions, and blockchain entries. Below is a breakdown of how the system's data is stored and processed.

3.3.1 Database Structure

- **User Data:** Stored in MongoDB, with attributes such as UserID, Name, Profile Information, Ratings, Reviews, and BlockchainID.
- **Product Data:** Product listings are also stored in MongoDB, with attributes such as ProductID, Name, Description, Price, Duration, and BlockchainToken.
- **Transaction Data:** Transactions are stored in MongoDB and linked to blockchain records. Each transaction stores the TransactionID, ProductID, RenterID, OwnerID, and payment details.

3.3.2 Blockchain Data

- **Ownership Data:** Product ownership data is stored on the Ethereum blockchain. The ownership history and product serial numbers are linked to blockchain tokens.
- **Transaction History:** The blockchain ledger stores the complete transaction history of each product, ensuring transparency and immutability.

3.3.3 Data Flow

The flow of data is crucial in RentEase. When a user rents a product, the following data flow occurs:

1. **User initiates rental:** User data is retrieved and authenticated.
2. **Product details:** The system fetches product data and checks availability.
3. **Transaction is processed:** The transaction is recorded in MongoDB, and ownership is validated through the blockchain

3.4 Databases and Storage Items

- **MongoDB:** Used for managing user data, product listings, and transaction records.
- **Firebase:** Primarily used for real-time data synchronization, especially for chat and real-time tracking.
- **Ethereum Blockchain:** Used for decentralized ownership and transaction history management.

Bibliography

- [1] Scikit-Learn Community. Machine learning based government id verification. <https://scikit-learn.org/stable/>, 2024.
- [2] Socket.IO Community. Socket.io documentation. <https://socket.io/docs/v4/>, 2024.
- [3] Tesseract OCR Community. Tesseract ocr documentation. <https://tesseract-ocr.github.io/>, 2024.
- [4] Ethereum Foundation. Decentralized identity verification using blockchain technology. *Blockchain Journal*, 2022.
- [5] Adam Geitgey. Face recognition python library. <https://pypi.org/project/face-recognition/>, 2024.
- [6] Karkavelraja J. Amazon sales dataset. <https://www.kaggle.com/datasets/karkavelrajaj/ama> 2024.
- [7] Making Lyst. Lightfm: A python implementation of a number of popular recommendation algorithms for both implicit and explicit feedback. <https://making.lyst.com/lightfm/docs>, 2024.
- [8] Daniel Schettler. Recsys 2020 e-commerce dataset. <https://www.kaggle.com/datasets/dschettler8845/recsys-2020-ecommerce-dataset>, 2024.
- [9] ChatterBot Team. Chatterbot: Machine learning based conversational dialog engine. <https://chatterbot.readthedocs.io/en/stable>, 2024.
- [10] Gift University. Cnic card detection dataset. <https://universe.roboflow.com/gift-university-qazvo/cnic-card-detection/dataset> 2024.